



Trabajo Práctico N°1

93.54 - Métodos Numéricos

Grupo N°1

Titular de cátedra: Alejandro Hayes

Legajo N°	Nombre
61665	Albertina Galan
62012	Santiago Feldman
61427	Gonzalo Ezequiel Linares
61467	Damián Ezequiel Sergi
61885	Agustín Luis Gullino

27 de junio de 2023

Índice

1. Ejercicio 1 - Sistemas de ecuaciones lineales	2
A. Eliminación Gaussiana	2
B. Factorización LU	3
C. Factorización QR	4
D. Método iterativo de Jacobi	5
E. Método iterativo de Gauss-Seidel	6
F. Método SOR	7
2. Ejercicio 2 - Factorización Cholesky	7
3. Ejercicio 3 - Regresión Lineal	8
4. Ejercicio 4 - Bisección	9
5. Ejercicio 5 - Punto fijo / Newton-Raphson	10

1. Ejercicio 1 - Sistemas de ecuaciones lineales

Dado el Sistema de Ecuaciones Lineales $AX = B$ con:

$$A = \begin{pmatrix} 3 & -1 & 0 & 0 & 0 \\ 0 & 5 & 0 & -1 & 2 \\ 1 & -1 & 9 & 0 & 0 \\ -2 & 3 & -1 & 12 & 1 \\ 0 & 0 & -1 & 1 & 15 \end{pmatrix} \quad B = \begin{pmatrix} -1 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

- Realizar un programa para resolverlo por eliminación de Gauss y Substitución Regresiva.
- Realizar un programa para resolverlo por factorización LU y Substitución Regresiva y Progresiva según corresponda.
- Realizar un programa para resolverlo por el método de Jacobi con un error menor que 10^{-6} .
- Realizar un programa para resolverlo por el método de Gauss-Seidel con un error menor que 10^{-6} .
- Realizar un programa para resolverlo por el método SOR con un error menor que 10^{-6} .

A. Eliminación Gaussiana

A continuación se presenta el código que implementa el algoritmo de eliminación Gaussiana para solucionar el sistema propuesto.

```

1 % El sistema a resolver es Ax=b dado por las siguientes matrices
2 A = [3, -1, 0, 0, 0;
3     0, 5, 0, -1, 2;
4     1, -1, 9, 0, 0;
5     -2, 3, -1, 12, 1;
6     0, 0, -1, 1, 15];
7
8 b = [-1; 0; 1; 1; 0];
9
10 % El sistema a resolver
11 disp("El sistema a resolver:")
12 [n, m] = size(A);
13 for i = 1:n
14     equation = "";
15     for j = 1:m-1
16         equation = strcat(equation, num2str(A(i, j)), "x", num2str(j), " + ");
17     end
18     equation = strcat(equation, num2str(A(i, m)), "x", num2str(m), " = ", num2str(b(i)));
19     disp(equation);
20 end
21
22 % Eliminacion Gaussiana
23 function x = gaussianElimination(A, b)
24     n = length(b);
25
26     % Check determinant
27     if det(A) == 0
28         error("The determinant of matrix A is zero. The system may not have a unique solution.");
29     end
30
31     % Forward elimination
32     for k = 1:n-1
33         for i = k+1:n
34             factor = A(i, k) / A(k, k);
35             A(i, :) = A(i, :) - factor * A(k, :);
36             b(i) = b(i) - factor * b(k);
37         end

```

```

38     end
39
40     % Back substitution
41     x = zeros(n, 1);
42     x(n) = b(n) / A(n,n);
43     for i = n-1:-1:1
44         x(i) = (b(i) - A(i,i+1:n) * x(i+1:n)) / A(i,i);
45     end
46
47 endfunction
48
49 % 1a) Resolver por eliminacion Gaussiana
50 x = gaussianElimination(A, b);
51
52 disp("\nCon Eliminacion Gaussiana:")
53 disp("x = ");
54 disp(x);

```

El resultado obtenido con este método es:

$$X = \begin{pmatrix} -0,33174 \\ 0,0047917 \\ 0,14850 \\ 0,038611 \\ 0,0073262 \end{pmatrix}$$

B. Factorización LU

A continuación se presenta el código que implementa el algoritmo de factorización LU para solucionar el sistema propuesto.

```

1 % El sistema a resolver es Ax=b dado por las siguientes matrices
2 A = [3, -1, 0, 0, 0;
3      0, 5, 0, -1, 2;
4      1, -1, 9, 0, 0;
5      -2, 3, -1, 12, 1;
6      0, 0, -1, 1, 15];
7
8 b = [-1; 0; 1; 1; 0];
9
10 % El sistema a resolver
11 disp("El sistema a resolver:")
12 [n, m] = size(A);
13 for i = 1:n
14     equation = "";
15     for j = 1:m-1
16         equation = strcat(equation, num2str(A(i, j)), "x", num2str(j), " + ");
17     end
18     equation = strcat(equation, num2str(A(i, m)), "x", num2str(m), " = ", num2str(b(i)));
19     disp(equation);
20 end
21
22 % Con descomposicion LU
23 function x = LU(A, b)
24     % LU factorization
25     n = length(b);
26     L = eye(n);
27     U = A;
28
29     for k = 1:n-1
30         for i = k+1:n
31             factor = U(i, k) / U(k, k);
32             L(i, k) = factor;
33             U(i, k:n) = U(i, k:n) - factor * U(k, k:n);
34         end
35     end
36
37     % Forward substitution

```

```

38 y = zeros(n, 1);
39
40 for i = 1:n
41     y(i) = b(i) - L(i, 1:i-1) * y(1:i-1);
42 end
43
44 % Back substitution
45 x = zeros(n, 1);
46
47 for i = n:-1:1
48     x(i) = (y(i) - U(i, i+1:n) * x(i+1:n)) / U(i, i);
49 end
50
51 endfunction
52
53 % 1b) Resolver con factorizacion LU
54 x = LU(A, b);
55
56 disp("Con factorizacion LU:")
57 disp("x = ");
58 disp(x);

```

El resultado obtenido con este método es:

$$X = \begin{pmatrix} -0,33174 \\ 0,0047917 \\ 0,14850 \\ 0,038611 \\ 0,0073262 \end{pmatrix}$$

C. Factorización QR

A continuación se presenta el código que implementa el algoritmo de factorización QR para solucionar el sistema propuesto.

```

1 % El sistema a resolver es Ax=b dado por las siguientes matrices
2 A = [3, -1, 0, 0, 0;
3     0, 5, 0, -1, 2;
4     1, -1, 9, 0, 0;
5     -2, 3, -1, 12, 1;
6     0, 0, -1, 1, 15];
7
8 b = [-1; 0; 1; 1; 0];
9
10 % El sistema a resolver
11 disp("El sistema a resolver:")
12 [n, m] = size(A);
13 for i = 1:n
14     equation = "";
15     for j = 1:m-1
16         equation = strcat(equation, num2str(A(i, j)), "x", num2str(j), " + ");
17     end
18     equation = strcat(equation, num2str(A(i, m)), "x", num2str(m), " = ", num2str(b(i)));
19     disp(equation);
20 end
21
22 % Con descomposicion QR
23 function x = QR(A, b)
24     % QR factorization
25     [m, n] = size(A);
26     Q = zeros(m, n);
27     R = zeros(n);
28
29     for j = 1:n
30         v = A(:, j);
31         for i = 1:j-1
32             R(i, j) = Q(:, i)' * A(:, j);
33             v = v - R(i, j) * Q(:, i);

```

```

34     end
35     R(j, j) = norm(v);
36     Q(:, j) = v / R(j, j);
37 end
38
39 % Back substitution
40 y = Q' * b;
41
42 n = size(R, 2);
43 x = zeros(n, 1);
44
45 for i = n:-1:1
46     x(i) = (y(i) - R(i, i+1:n) * x(i+1:n)) / R(i, i);
47 end
48 endfunction
49
50 % 1c) Resolver con factorizacion QR
51 x = QR(A, b);
52
53 disp("Con factorizacion QR:")
54 disp("x = ");
55 disp(x);

```

El resultado obtenido con este método es:

$$X = \begin{pmatrix} -0,33174 \\ 0,0047917 \\ 0,14850 \\ 0,038611 \\ 0,0073262 \end{pmatrix}$$

D. Método iterativo de Jacobi

A continuación se presenta el código que implementa el algoritmo de Jacobi para solucionar el sistema propuesto.

```

1 % El sistema a resolver es Ax=b dado por las siguientes matrices
2 A = [3, -1, 0, 0, 0;
3      0, 5, 0, -1, 2;
4      1, -1, 9, 0, 0;
5      -2, 3, -1, 12, 1;
6      0, 0, -1, 1, 15];
7
8 b = [-1; 0; 1; 1; 0];
9
10 % El sistema a resolver
11 disp("El sistema a resolver:")
12 [n, m] = size(A);
13 for i = 1:n
14     equation = "";
15     for j = 1:m-1
16         equation = strcat(equation, num2str(A(i, j)), "x", num2str(j), " + ");
17     end
18     equation = strcat(equation, num2str(A(i, m)), "x", num2str(m), " = ", num2str(b(i)));
19     disp(equation);
20 end
21
22 % Con el metodo iterativo de Jacobi
23 function x = Jacobi(A, b, maxIt, error)
24     n = length(b);
25     x = zeros(n, 1);
26     xPrev = x;
27
28     for iteration = 1:maxIt
29         for i = 1:n
30             sum = 0;
31             for j = 1:n
32                 if i ~= j

```

```

33         sum = sum + A(i, j) * xPrev(j);
34     end
35     end
36     x(i) = (b(i) - sum) / A(i, i);
37 end
38
39     if norm(x - xPrev) < error
40         break;
41     end
42
43     xPrev = x;
44 end
45
46 endfunction
47
48 % 1d) Resolver con el metodo de Jacobi con Error<10^-6
49 x = Jacobi(A, b, 1000, 1e-06);
50
51 disp("Con el metodo de Jacobi:")
52 disp("x = ");
53 disp(x);

```

El resultado obtenido con este método es:

$$X = \begin{pmatrix} -0,33174 \\ 0,0047917 \\ 0,14850 \\ 0,038611 \\ 0,0073262 \end{pmatrix}$$

E. Método iterativo de Gauss-Seidel

A continuación se presenta el código que implementa el algoritmo de Gauss-Seidel para solucionar el sistema propuesto.

```

1 A = [3, -1, 0, 0, 0;
2       0, 5, 0, -1, 2;
3       1, -1, 9, 0, 0;
4       -2, 3, -1, 12, 1;
5       0, 0, -1, 1, 15];
6
7 b = [-1, 0, 1, 1, 0];
8
9 function x = GaussSeidelMethod(A, b)
10
11     n = size(A, 1);
12     x = zeros(n, 1);
13
14     iteration = 0;
15     error = Inf;
16
17     while error > 10e-6
18         xPrev = x;
19         for i = 1:n
20             x(i) = (b(i) - A(i, 1:i-1)*x(1:i-1) - A(i, i+1:end)*xPrev(i+1:end)) / A(i, i)
21             ;
22         end
23         error = norm(x - xPrev, inf);
24         iteration = iteration + 1;
25     end
26
27 x = GaussSeidelMethod(A, b);
28 disp("Solution:"); disp(x)

```

El resultado obtenido con este método es:

$$X = \begin{pmatrix} -0,33174 \\ 0,0047919 \\ 0,14850 \\ 0,038611 \\ 0,0073262 \end{pmatrix}$$

F. Método SOR

Por último, resolvimos el sistema por el método SOR.

```

1
2 A = [3, -1, 0, 0, 0; 0, 5, 0, -1, 2; 1, -1, 9, 0, 0; -2, 3, -1, 12, 1; 0, 0, -1, 1, 15];
3 b = [-1; 0; 1; 1; 0];
4 omega = 1.1;           %W
5 maxIter = 1000;        % Maximum number of iterations
6 tol = 1e-6;           % Error
7
8 function [x, iter] = sor(A, b, omega, maxIter, tol)
9
10     n = size(A, 1);
11     x = zeros(n, 1);
12     iter = 0;
13     error = tol + 1;
14
15     while (error > tol) && (iter < maxIter)
16         xPrev = x;
17         for i = 1:n
18             sum1 = A(i, 1:i-1) * x(1:i-1);
19             sum2 = A(i, i+1:n) * xPrev(i+1:n);
20             x(i) = (1 - omega) * xPrev(i) + (omega / A(i, i)) * (b(i) - sum1 - sum2);
21         endfor
22
23         error = norm(x - xPrev) / norm(x);
24         iter = iter + 1;
25     endwhile
26 endfunction
27
28 [x, iter] = sor(A, b, omega, maxIter, tol);
29
30 disp('Solution:');
31 disp(x);
32 disp('Number of iterations:');
33 disp(iter);

```

El resultado que obtuvimos fue:

$$X = \begin{pmatrix} -0.3317360 \\ 0.0047917 \\ 0.1485031 \\ 0.0386108 \\ 0.0073261 \end{pmatrix}$$

2. Ejercicio 2 - Factorización Cholesky

Dado el Sistema de ecuaciones lineales $AX = B$ donde:

$$A = \begin{pmatrix} 4 & 1 & 2 \\ 1 & 2 & 0 \\ 2 & 0 & 5 \end{pmatrix} \quad B = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}$$

Se pide realizar un programa que haga la factorización de Cholesky de A y resuelva el sistema por sustitución Regresiva y Progresiva según corresponda.

Se implementó el siguiente código para resolver el sistema:

```

1 A = [4, 1, 2;
2     1, 2, 0;
3     2, 0, 5];
4
5 B = [-1; 0; 1];
6
7 % Get the Cholesky coefficients
8 g11 = sqrt(A(1,1));
9 g21 = A(2,1)/g11;
10 g31 = A(3,1)/g11;
11 g22 = sqrt(A(2,2)-g21^2);
12 g32 = (A(3,2)-g21*g31)/g22;
13 g33 = sqrt(A(3,3)-g31^2-g32^2);
14
15 G = [g11, 0, 0;
16     g21, g22, 0;
17     g31, g32, g33];
18
19 GT = transpose(G);
20
21 n = size(GT, 1);
22 Y = zeros(n, 1);
23 X = zeros(n, 1);
24
25 % Forward substitution
26 for i = 1:n
27     Y(i) = (B(i) - G(i, 1:i-1)*Y(1:i-1)) / G(i, i);
28 end
29
30 % Back substitution
31 for i = n:-1:1
32     X(i) = (Y(i) - GT(i, i+1:end)*X(i+1:end)) / GT(i, i);
33 end
34
35
36 disp("X="), disp(X)

```

El código da como resultado:

$$X = \begin{pmatrix} -0,518518518518518 \\ 0,259259259259259 \\ 0,407407407407407 \end{pmatrix} = \frac{1}{27} \cdot \begin{pmatrix} -14 \\ 7 \\ 11 \end{pmatrix}$$

3. Ejercicio 3 - Regresión Lineal

Un investigador reporta los datos tabulados a continuación, de un experimento para determinar la tasa de crecimiento de bacterias k , como función de la concentración de oxígeno x . Se sabe que dichos datos pueden modelarse por medio de la siguiente ecuación:

$$k(x) = k_{max} \frac{x^2}{x^2 + c_s}$$

donde c_s y k_{max} son parámetros. Use una transformación para hacer lineal esta ecuación. Después utilice regresión lineal para estimar c_s y k_{max} , y pronostique la tasa de crecimiento para $x = 2 \text{ mg/L}$.

x	0,5	0,8	1,5	2,5	4
k	1,1	2,4	5,3	7,6	8,9

Para la resolución se aplicó la transformación $x^2 = 1/u$, llegando a la ecuación de la recta $\frac{1}{k_{max}} + \frac{c_s}{k_{max}} * u$. transformando los datos y utilizando el método de cuadrados mínimos se estimaron los parámetros $k_{max} = 10.061$ y $c_s = 2.0372$. Con esto se calculó el valor $k(2) = 6.665843$.

El código utilizado fue el siguiente:

```

1 x = [0.5 0.8 1.5 2.5 4];
2 k = [1.1 2.4 5.3 7.6 8.9];
3
4 %{
5 k(x) = kmax * x**2 / (x**2 + cs) ->
6 f(x) = 1/k(x) = 1/kmax * (x**2 + cs) / x**2
7
8 x -> u, x > 0
9 x**2 = 1/u
10
11 f(u) = 1/kmax + cs/kmax*u
12 %}
13 f = 1./k;
14 u = 1./(x.*x);
15
16
17 %{
18 A'*x=A'*b
19 x = [cs/kmax 1/kmax]
20 %}
21 A = [u' ones(columns(u), 1)];
22 y = f';
23
24 params = linsolve(A'*A, A'*y);
25 disp("Los parametros estimados son: ")
26 kmax = 1/params(2)
27 cs = params(1)*kmax
28 disp("Ahora reemplazo en la funcion con los parametros reemplazados con c = 2:")
29 c = 2;
30 printf("k(2) = %f \n", kmax*c^2/(c^2+cs))

```

4. Ejercicio 4 - Bisección

Dada la siguiente ecuación:

$$e^{-x^2} = 1 - \frac{1}{x^2 + 1}$$

- Hallar algún intervalo que contenga a la solución.
- Realizar un programa que permita resolver el problema por el método de bisección y resuelva el problema con un error menor que 10^{-6} .

Se realizaron los cálculos pertinentes y se encontró que el intervalo $[0,1]$ contenía la solución del problema. Luego, se procedió a realizar el programa que lo resuelve por el método de bisección. De este último se obtuvo que la solución es $x = \mathbf{0.89803}$ con un error menor a 10^{-6} . El código que se utilizó se puede encontrar a continuación:

```

1
2 a = 0;
3 b = 1;
4 tol = 1e-6;           % Error
5
6 function val = fun(x)
7     val = exp(-x^2) + 1/(x^2+1) - 1;
8 endfunction
9
10 function [sol, niter, error] = biseccion(a,b,cotaerror)
11
12     cont=0;

```

```

13  xant=a;
14  xsig=b;
15
16  while abs(xsig-xant) > cotaerror
17      cont=cont+1;
18      if fun(xant)*fun(xsig)<0
19          xmed=(xant+xsig)/2;
20          if fun(xmed)*fun(xant)<0
21              xsig=xmed;
22          elseif fun(xsig)*fun(xmed)<0
23              xant=xmed;
24          else
25              sol=xmed;
26              error=0;
27              niter=cont;
28              break
29          end
30      elseif fun(xant)*fun(xsig)>0
31          disp('Intervalo Incorrecto')
32          break
33      elseif fun(xant)==0
34          sol=xant;
35          error=0;
36          niter=cont;
37          break
38      else
39          sol=xsig;
40          error=0;
41          niter=cont;
42          break
43      end
44  end
45
46  sol=xmed;
47  niter=cont;
48  error=abs(xsig-xant);
49 endfunction
50
51 [sol, iter, error] = biseccion(a,b,tol);
52
53 disp('Solution:');
54 disp(sol);
55 disp('Number of iterations:');
56 disp(iter);
57 disp('Error:');
58 disp(error);

```

5. Ejercicio 5 - Punto fijo / Newton-Raphson

La ecuación

$$c(t) = c_e \left(1 + e^{-0,04t}\right) + c_o e^{-0,06t}$$

permite calcular la concentración de un químico en un reactor donde se tiene una mezcla completa. Si la concentración inicial es $c_0 = 5$ y la concentración de entrada es $c_e = 12$, calcule el tiempo requerido para que c sea el 85 % de c_e con una cota de error menor a 10^{-6} . Para ello:

- Hallar algún intervalo que contenga a la solución.
- Realizar un programa que permita resolver el problema por el método de punto fijo.
- Realizar un programa que permita resolver el problema por el método de Newton-Raphson.

```

1 %La función en la cual se aplica el método de punto fijo es f(t)=t-(12*(1-e^(-0.04t))+5*e
  ^(-0.06t)-10.2)
2
3 %Para encontrar un intervalo en donde se encuentre la solución basta con

```

```

4 %evaluar  $f(t) = 12*(1-e^{(-0.04*(t))})+5*e^{(-0.06*(t))}-10.2$ ; (donde  $10.2 = 0.85*Ce$ )
5 %y encontrar dos puntos x1, x2 tal que  $f(x1)*f(x2)<0$ . Tales podrían ser por ejemplo
6 %x1=0 y x2=50
7 %Es válido que la raíz sea única en este punto ya que  $f'(t)>0$  para todo  $t>0$ 
8 %Por lo que la función es monótona creciente y la raíz es única
9
10
11 format long;
12 function y=g(t) %Funcion para el metodo de punto fijo
13     y = (t)-(12*(1-e^{(-0.04*(t))})+5*e^{(-0.06*(t))}-10.2);
14 endfunction
15
16 function y=f(t) %Funcion para el metodo de Newton-Raphson
17     y = 12*(1-e^{(-0.04*(t))})+5*e^{(-0.06*(t))}-10.2;
18 endfunction
19
20 function y=df(t) %Funcion para el metodo de Newton-Raphson
21     y = (0.04)*12*e^{(-0.04*(t))}+(-0.06)*5*e^{(-0.06*(t))};
22 endfunction
23
24 %Parametros para el método de punto fijo
25 x0FP = 25; %Tomo el punto medio del intervalo previamente definido
26 cotaErrorFP = 10^{(-6)};
27
28 %Funcion para el método de punto fijo
29 xAntFP = x0FP;
30 xSigFP = g(xAntFP);
31 contFP = 1;
32 while abs(xSigFP-xAntFP)>cotaErrorFP
33     xAntFP=xSigFP;
34     xSigFP=g(xAntFP);
35     contFP=contFP+1;
36 end
37
38 %Soluciones para el método de punto fijo
39 disp("Las soluciones con el método del punto fijo son:")
40 solFP=xSigFP
41 nIterFP=contFP
42 errorFP=abs(xSigFP-xAntFP)
43
44 %Parametros para el método de Newton Raphson
45 x0NR = 25; %Tomo el punto medio del intervalo previamente definido
46 cotaErrorNR = 10^{(-6)};
47
48 %Funcion para el método de Newton Raphson
49 xAntNR = 0;
50 xSigNR = x0NR;
51 contNR = 0;
52 while abs(xSigNR-xAntNR)>cotaErrorNR
53     xAntNR=xSigNR;
54     xSigNR=xAntNR - ( f(xAntNR) / df(xAntNR) );
55     contNR=contNR+1;
56 end
57
58 %Soluciones para el método de Newton Raphson
59 disp("Las soluciones con el método de Newton Raphson son:")
60 solNR=xSigNR
61 nIterNR=contNR
62 errorNR=abs(xSigNR-xAntNR)

```

El código da las siguientes líneas como resultado de la solución, número de iteraciones y el error con el método del punto fijo:

$$\begin{aligned}
 solFP &= 42.52791414477118 \\
 nIterFP &= 207 \\
 errorFP &= 9.769021787064958e-07
 \end{aligned}$$

El código da las siguientes líneas como resultado de la solución, número de iteraciones y el error con el método del Newton Raphson:

$solNR = 42.52792838324716$
 $nIterNR = 5$
 $errorNR = 1.211017774949141e - 08$